LLARS - Script Creator Library

Current Creator API Recipes

Welcome to the LLARS copy/paste reference for current script creators.

This library is a companion to the Script Creator & Framework Guide. The guide explains the framework model; this document focuses on current Creator API recipes you can paste and adapt.

Traditional IniRead/Random/Send/label-SetTimer patterns are intentionally moved to the separate Legacy Script Creator Guide so this library can stay clean and API-first.

Table of Contents

1. How to Use This Library
2. Config.ini Building Blocks
3. RunCount Recipes
4. Click Recipes
5. Keyboard Recipes
6. Status Recipes
7. Sleep and Runtime Recipes
8. Random Sleep Recipes
9. Optional Feature Recipes
10. Typed Config Read Recipes
11. One-Shot Timer Recipes
12. Repeating Timer Recipes
13. Bound Callback Recipes
14. Timer Script Recipe
15. MultiTimer Recipe
16. Single Color Detection
17. Multi Color Detection
18. Prevent Repeated Pixel Triggers
19. Dynamic Coordinates and NaturalClick
20. Custom Raw Game-Facing Actions
21. Common Action Combinations
22. Developer Mode Testing Recipes
23. Conversion Mappings
24. Patterns to Avoid
25. Creator API Cheat Sheet
26. The Simple Version

Input Backend

Creator API input helpers automatically use AutoHotInterception (AHI), the recommended LLARS backend, when active. No AHI-specific creator code is required; the same helpers use Standard AutoHotkey only when LLARS is in fallback mode.

1. How to Use This Library

Most current LLARS scripts can be built from a few calls: perform an action, wait using a named Config.ini section, let LLARS own the lifecycle, and read configuration directly only when task logic needs a value.

```
Run(ctx)
{
    LLARS_SetStatus("Banking")
    LLARS_Click("Bank")
    LLARS_Sleep("Sleep Short")
    LLARS_PressHotkey("Bank Preset")
    LLARS_Sleep("Final Sleep", true)
}
```

> **Note:** If you are about to write IniRead + Random + Click/Sleep for a normal configured action, stop and check whether LLARS_Click(), LLARS_Sleep(), LLARS_PressHotkey(), or a typed config helper already does it.

2. Config.ini Building Blocks

Area click

```
#Area the script should click.
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

Exact point

```
#Exact pixel the script should monitor.
[Pixel Coordinate]
type=coordinate
x=
y=
```

Hotkey

```
#RuneScape hotkey for this action.
[Action Hotkey]
type=hotkey
hotkey=
```

Color

```
#Color that represents the desired state.
[Target Color]
type=color
coordinate=Pixel Coordinate
color=0xFFFFFF
```

Sleep/timer

```
[Sleep Short]
min=500
max=1000

[Action Timer]
min=10000
max=30000
```

Optional typed setting

```
[Optional Hotkey]
option=false
type=hotkey
hotkey=
```

3. RunCount Recipes

Basic

```
Start:

LLARS_RunCount(Func("Run"))

return

Run(ctx)
{
    LLARS_Click("Action Location")
    LLARS_Sleep("Final Sleep", true)
}
```

Different first run

```
Run(ctx)
{
    if (ctx.IsFirst)
    {
        LLARS_Click("Prime Location")
    }
    else
    {
        LLARS_Click("Normal Location")
    }

    LLARS_Sleep("Final Sleep", true)
}
```

Last run special action

```
if (ctx.IsLast)
    LLARS_SetStatus("Finishing")
```

Progress-aware creator logic

```
Log("PROGRESS", "Run " . ctx.Iteration . "/" . ctx.Total)
remaining := ctx.Remaining
```

4. Click Recipes

Left click configured area

```
LLARS_Click("Action Location")
```

Right click

```
LLARS_Click("Action Location", "right")
```

Two clicks with travel wait

```
LLARS_Click("First Location")
LLARS_Sleep("Travel Time")
LLARS_Click("Second Location")
```

Optional click

```
if LLARS_ConfigEnabled("Optional Click")
    LLARS_Click("Optional Click")
```

LLARS_Click() handles fixed points and rectangular random ranges. NaturalClick performs the movement and will not physically click until the cursor reaches the exact selected target pixel.

5. Keyboard Recipes

Configured hotkey

```
LLARS_PressHotkey("Bank Preset")
```

Purposeful hardcoded key

```
LLARS_PressKey("Space")
LLARS_PressKey("Enter")
LLARS_PressKey("4")
```

Mouse wheel

```
LLARS_PressKey("WheelDown")
```

Click then fixed key

```
LLARS_Click("Action Location")
LLARS_Sleep("Sleep Short")
LLARS_PressKey("Space")
```

> **Note:** Do not make a key configurable just because it is hardcoded. Use LLARS_PressKey() when the fixed key is purposeful script behavior; use LLARS_PressHotkey() when the user is expected to choose the binding.

6. Status Recipes

```
LLARS_SetStatus("Banking")
LLARS_SetStatus("Waiting", "Portable")
LLARS_SetStatus("Cutting", "Refined Plank")
```

Status is useful around longer sequences and makes Developer Mode and the main GUI easier to follow.

7. Sleep and Runtime Recipes

Ordinary configured wait

```
LLARS_Sleep("Sleep Short")
```

True final RunCount wait

```
LLARS_Sleep("Sleep Craft", true)
```

First/following timing

```
if (ctx.IsFirst)
{
    LLARS_Sleep("First Setup Sleep")
}
else
{
    LLARS_Sleep("Normal Travel Sleep")
}

LLARS_Sleep("Final Sleep", true)
```

- Keep these calls inside the exact editing markers.
- Use literal section names when the pre-run estimator needs to recognize them.
- The current-loop countdown becomes exact at the final=true sleep.

8. Random Sleep Recipes

Normal use

```
LLARS_RandomSleep()
```

Know the roll before later logic

```
willSleep := LLARS_RandomSleepRoll()

if (willSleep)
    LLARS_SetStatus("Random Sleep Pending")

LLARS_RandomSleep()
```

LLARS_RandomSleep() consumes a prepared roll if one exists; otherwise it rolls when called. The RunCount estimate uses the probability-weighted expected contribution and adjusts the current loop to what actually happened.

9. Optional Feature Recipes

Simple boolean

```
[Feature]
option=false

if LLARS_ConfigReadBool("Feature")
{
    ; feature behavior
}
```

Optional configured hotkey

```
[Special Action]
option=false
type=hotkey
hotkey=

if LLARS_ConfigEnabled("Special Action")
    LLARS_PressHotkey("Special Action")
```

Dependent setting

```
[Cannon]
option=false

[Cannon Restock]
depends=Cannon
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

10. Typed Config Read Recipes

Integer with minimum

```
count := LLARS_ConfigReadInteger("Settings", "count", "", 1)
```

Number with range

```
offset := LLARS_ConfigReadNumber("Settings", "offset", 0, -50, 50)
```

Required text

```
name := LLARS_ConfigReadString("Settings", "name", "", true)
```

Choice

```
mode := LLARS_ConfigReadChoice("Settings", "mode", "One|Two|Three", "One")
```

Resolved point

```
if LLARS_ConfigReadPoint("Action Location", x, y)
{
    ; task-specific calculations using x/y
}
```

Configured range

```
if LLARS_ConfigReadRange("Action Timer", minValue, maxValue)
{
    ; task-specific use of the range
}
```

11. One-Shot Timer Recipes

```
ActionTimerID := LLARS_TimerOnce("Action Timer", Func("Action"))

Action()
{
    LLARS_PressKey("Space")
}
```

A one-shot callback is removed after it fires. If the run ends before it fires, framework cleanup removes it.

12. Repeating Timer Recipes

```
ActionTimerID := LLARS_TimerRepeat("Action Timer", Func("Action"))

Action()
{
    LLARS_Click("Action Location")
}
```

Every repeat receives a newly randomized interval from the timer section. This is the preferred replacement for manual randomize/reschedule label timers.

Stop it early

```
LLARS_TimerStop(ActionTimerID)
```

13. Bound Callback Recipes

Use Func().Bind() when one callback should serve several configured actions.

```
ScheduleAction(section, statusText)
{
    return LLARS_TimerOnce(section, Func("ActionTimer").Bind(section, statusText))
}

ActionTimer(section, statusText)
{
    LLARS_SetStatus(statusText)
    LLARS_PressHotkey(section)
}
```

This pattern is useful in scripts such as combat helpers where many optional sections share the same scheduling logic.

14. Timer Script Recipe

The maintained Timer template owns LLARS_StartTimerRun(), the overall countdown, EndMsg, and completion. Creator code normally supplies setup plus callbacks:

```
TimerSetup()
{
    global ScriptTimerID
    ScriptTimerID := LLARS_TimerRepeat("Script Timer", Func("ScriptTimer"))
}

ScriptTimer()
{
    LLARS_SetStatus("Working")
    LLARS_Click("Action Location")
}
```

Note: Timer templates prompt for an overall duration. For task-defined completion, use the UntilDone lifecycle (LLARS_SCRIPT_TYPE := "UntilDone" plus start/end helpers); no duration or run-count prompt is used.

15. MultiTimer Recipe

```
MultiTimerSetup()
{
    global TimerA, TimerB, TimerC

    TimerA := LLARS_TimerRepeat("Timer A", Func("ActionA"))
    TimerB := LLARS_TimerRepeat("Timer B", Func("ActionB"))
    TimerC := LLARS_TimerRepeat("Timer C", Func("ActionC"))
}

ActionA()
{
    LLARS_PressHotkey("Hotkey A")
```

```
}

ActionB()
{
    LLARS_Click("Location B")
}

ActionC()
{
    LLARS_PressKey("Space")
}
```

16. Single Color Detection

```
CheckPixel()
{
    global CheckPixelID, ResetCheckID

    if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
    {
        LLARS_TimerStop(CheckPixelID)
        LLARS_Sleep("Sleep Timer")

        if !LLARS_RunActive()
            return

        LLARS_Click("Action Location")
        ResetCheckID := LLARS_TimerRepeat("Pixel Check Timer", Func("ResetCheck"))
    }
}
```

The API handles the pixel read and focus safety. The callback only needs task-specific trigger/reset behavior.

17. Multi Color Detection

```
ColorSections := ["Target Color One", "Target Color Two", "Target Color Three"]
matched := LLARS_PixelMatchesAny("Pixel Coordinate", ColorSections)

if (matched != "")
{
    LLARS_SetStatus("Detected", matched)
    LLARS_Click("Action Location")
}
```

Rotate matching action locations

```
LocationSections := ["Action Location One", "Action Location Two"]
locationIndex := 1

locationSection := LocationSections[locationIndex]
LLARS_Click(locationSection)

locationIndex++
if (locationIndex > LocationSections.Length())
    locationIndex := 1
```

18. Prevent Repeated Pixel Triggers

After one match is handled, stop detection until the watched pixel leaves every configured target state.

```
ResetCheck()
{
    global CheckPixelID, ResetCheckID, ColorSections

    if (LLARS_PixelMatchesAny("Pixel Coordinate", ColorSections) = "")
    {
        LLARS_TimerStop(ResetCheckID)
        CheckPixelID := LLARS_TimerRepeat("Pixel Check Timer", Func("CheckPixel"))
    }
}
```

This keeps one unchanged screen state from producing repeated clicks.

19. Dynamic Coordinates and NaturalClick

Use LLARS_Click() for normal Config.ini coordinate sections. Use NaturalClick only when creator logic has already calculated the final destination.

```
x := baseX + offsetX
y := baseY + offsetY
NaturalClick(x, y)
```

NaturalClick caps each calculated movement path at 3000 ms, but has no fixed retry or total elapsed-time limit.
If the user moves the cursor, LLARS keeps correcting and clicks only after reaching the exact target pixel.

> Note: When Developer Mode is open and LLARS can identify the matching configured range, the active coordinate box is shown during the click and remains visible briefly after a successful click.

20. Custom Raw Game-Facing Actions

Sometimes a script needs an operation that has no dedicated Creator API wrapper, such as a specialized MouseMove. Restore RuneScape immediately before the raw physical action.

```
if !LLARS_WaitForRuneScape("Custom MouseMove")
    return

MouseMove, %x%, %y%, %speed%
```

If more time-consuming logic occurs before the physical operation, call LLARS_WaitForRuneScape() again immediately before dispatch. LLARS_Sleep() restores RuneScape when it finishes; raw Sleep does not.

21. Common Action Combinations

Bank -> preset -> action -> fixed key

```
LLARS_SetStatus("Banking")
LLARS_Click("Bank")
LLARS_Sleep("Sleep Short")
LLARS_PressHotkey("Bank Preset")
LLARS_Sleep("Sleep Short")
```

```
LLARS_Click("Action Location")
LLARS_Sleep("Sleep Travel")
LLARS_PressKey("Space")
```

First run prime, later runs normal

```
if (ctx.IsFirst)
    LLARS_Click("Bank Prime")
else
    LLARS_Click("Bank Main")

LLARS_PressHotkey("Bank Preset")
LLARS_Sleep("Final Sleep", true)
```

Optional timed hotkey

```
if LLARS_ConfigReadBool("Potion")
    PotionTimerID := LLARS_TimerRepeat("Potion", Func("PotionAction"))

PotionAction()
{
    LLARS_PressHotkey("Potion")
}
```

22. Developer Mode Testing Recipes

- Open Developer Mode before a click and confirm the coordinate range appears during the click and briefly after success.
- Move the mouse while NaturalClick is traveling and confirm it corrects without clicking early.
- For multi-color scripts, scroll Live Inspector and compare Current Pixel to every Pixel Target.
- For Timer, confirm Progress matches the main GUI; for UntilDone, verify its completion condition ends the run.
- Scroll Active Hotkeys and Live Inspector; both should preserve the user position while data refreshes.
- Switch to Lightweight Mode and confirm the Developer Mode hotkey toggles Lightweight until Full Mode is selected again.
- Alt-Tab during LLARS_Sleep() and confirm RuneScape is restored when the sleep finishes.

23. Conversion Mappings

Older pattern	Current replacement
Loop, %runcount% + Begin/EndLoop	LLARS_RunCount(Func("Run"))
firstrun global	ctx.IsFirst
IniRead coordinates + Random + NaturalClick	LLARS_Click("Section")
IniRead hotkey + Send	LLARS_PressHotkey("Section")
Purposeful Send, {Space}	LLARS_PressKey("Space")
IniRead min/max + Random + Sleep	LLARS_Sleep("Section")
Final randomized sleep + LLARS_FinalSleep	LLARS_Sleep("Section", true)
Randomized/rescheduled label timer	LLARS_TimerRepeat("Section", Func("Callback"))
PixelGetColor + configured color read	LLARS_PixelMatches(...)
Several sequential color comparisons	LLARS_PixelMatchesAny(...)

24. Patterns to Avoid

- Do not use raw Send/Click/MouseClick for normal actions that have Creator API helpers.
- Do not keep creator-owned runcount/firstrun lifecycle in new RunCount scripts.
- Do not use dynamic section names for RunCount sleeps you expect the pre-run estimator to resolve.
- Do not override CoordMode in normal scripts. Core owns coordinate mode.
- Do not copy NaturalClick or other Core functions into a script.
- Do not duplicate SCRIPT_EDIT markers.
- Do not run raw game-facing actions without LLARS_WaitForRuneScape() immediately before them.
- Do not use Developer Mode as a dependency for script behavior.

25. Creator API Cheat Sheet

```
LLARS_RunCount(Func("Run"))
LLARS_SCRIPT_TYPE := "UntilDone"; LLARS_StartUntilDoneRun(); LLARS_EndUntilDoneRun()
LLARS_Click("Coordinate Section")
LLARS_Click("Coordinate Section", "right")
LLARS_PressKey("Space")
LLARS_PressHotkey("Hotkey Section")
LLARS_SetStatus("Working")

LLARS_Sleep("Sleep Section")
LLARS_Sleep("Final Sleep", true)
LLARS_RandomSleep()

TimerID := LLARS_TimerOnce("Timer Section", Func("Action"))
TimerID := LLARS_TimerRepeat("Timer Section", Func("Action"))
LLARS_TimerStop(TimerID)
LLARS_TimerStopAll()

color := LLARS_PixelColor("Pixel Coordinate")
matched := LLARS_PixelMatches("Pixel Coordinate", "Target Color")
which := LLARS_PixelMatchesAny("Pixel Coordinate", ColorSections)

value := LLARS_ConfigRead("Section", "key", "default")
flag := LLARS_ConfigReadBool("Feature")
count := LLARS_ConfigReadInteger("Settings", "count", "", 1)
number := LLARS_ConfigReadNumber("Settings", "value", 0)
text := LLARS_ConfigReadString("Settings", "name", "", true)
choice := LLARS_ConfigReadChoice("Settings", "mode", "One|Two")
LLARS_ConfigReadPoint("Location", x, y)
LLARS_ConfigReadRange("Timer", minValue, maxValue)

if !LLARS_WaitForRuneScape("Custom action")
    return
```

26. The Simple Version

- Use the current API for the normal thing first.
- Use Config.ini section names instead of repeating IniRead boilerplate.
- Use LLARS_RunCount(Func("Run")) instead of owning the loop.
- Use UntilDone start/end helpers when task logic determines completion instead of a timer or run count.
- Use LLARS_PressKey for purposeful fixed keys and LLARS_PressHotkey for user-configurable bindings.
- Use managed callback timers for script actions.
- Use LLARS_PixelMatchesAny for one pixel with several target colors.
- Use LLARS_WaitForRuneScape immediately before unavoidable raw game-facing code.
- Use the Legacy Guide when you intentionally need traditional AHK methods.